Université Joseph Ki-Zerbo
IBAM, M1 Informatique

Virtualisation et Cloud Computing, Dr SOMDA

Journal de bord du projet RÉSULTATS-BF

Groupe 04 :

KOBENDE Pêgdwendé Evrard,
PORGO Abdoul Nassirou,
OUEDRAOGO Armand Hendrix,
THIOMBIANO Yempabou Romaric Johan

Table des matières

Répartition des rôles 3
Décisions prises et raisons 3
Difficultés rencontrées et solutions 4

Ce journal résume la répartition du travail, nos principales décisions et les difficultés rencontrées. Les détails se trouvent dans le dossier d'architecture, aux sections indiquées.

Répartition des rôles

PORGO Abdoul Nassirou s'est chargé du réseau et de la répartition de charge, avec l'extension E1. KOBENDE Pêgdwendé Evrard a développé l'application. OUEDRAOGO Armand Hendrix s'est occupé de la base de données, et THIOMBIANO Yempabou Romaric Johan des essais et de la supervision, avec l'extension E2. Comme le demande l'énoncé, chacun a aussi travaillé en dehors de son domaine : KOBENDE a écrit le scénario V2, PORGO le scénario V6 et la remise en service de la base, THIOMBIANO l'installation de node_exporter dans le socle commun, et OUEDRAOGO le script de vérification de la réplication (R5).

Décisions prises et raisons

Qui	Décision	Raison
Groupe	Réaliser les extensions E1 et E2	Ce sont les plus faciles à démontrer, et elles ne dépendent pas du code de l'application
PORGO	Utiliser HAProxy plutôt que Nginx	HAProxy sait interroger `/sante` de lui-même, alors que la version libre de Nginx ne fait que des contrôles passifs. Il sait aussi rejouer une requête et fournit des métriques
PORGO	Générer la configuration de HAProxy et de keepalived à partir de `plateforme.json`	Aucune adresse écrite en dur, et la même source que le Vagrantfile (R3)
PORGO	Créer le fichier `.env` au premier `vagrant up`, avec des mots de passe aléatoires	Respecter à la fois R1 (aucune intervention manuelle) et R7 (aucun secret dans le dépôt)
PORGO	VRRP en unicast, les deux répartiteurs démarrant en attente avec des priorités de 150 et 100	Ne pas dépendre du multicast et éviter d'avoir deux maîtres au démarrage
KOBENDE	Ouvrir une connexion à la base par requête, sans réserve de connexions	Garder une application strictement sans état, qui se reconnecte naturellement après une bascule
KOBENDE	Connexion à `bdd1,bdd2` avec `target_session_attrs=read-write`	L'application suit la bascule sans être modifiée ni redémarrée
KOBENDE	Service systemd sans droits root, configuration en mode 600, `connect_timeout=2`	Limiter les privilèges et garantir que `/sante` répond avant la fin du délai de la sonde
THIOMBIANO	Générer les cibles de Prometheus et rejouer le rôle obs à chaque `vagrant up`	app3 et app4 apparaissent seules quand on les ajoute (7, puis 9, puis 7 machines mesurées)
THIOMBIANO	Décrire le tableau de bord Grafana par des fichiers	Il est recréé à l'identique après un `vagrant destroy` (R1)
THIOMBIANO	Pour V3, trois tirs par configuration, valeur médiane et calcul du débit utile	Un seul tir est trop variable, et ab compte les réponses 503 dans son nombre de requêtes par seconde
Groupe	Ne pas ajouter de réserve de connexions à deux jours du rendu	Il aurait fallu refaire les mesures V2, V4, R4 et V6

Difficultés rencontrées et solutions

Qui	Difficulté	Solution
PORGO	La version `202404.23.0` de la box donnée dans l'énoncé n'existe pas pour VirtualBox	Nous avons fixé la version `202510.26.0` pour tout le groupe
PORGO	L'exportateur Prometheus de HAProxy n'était activé que sur lb2 : la commande de détection échouait au hasard à cause de l'option `pipefail`	La sortie de `haproxy -vv` est d'abord enregistrée dans une variable, puis testée
PORGO	La première mesure de V6 partait de l'ordre `vagrant halt`, alors que lb1 continue de servir pendant son arrêt	Nous mesurons maintenant l'interruption réelle, soit 1,7 s. Les deux fichiers de résultats sont conservés
PORGO	Une machine virtuelle s'est figée par manque de mémoire sur l'ordinateur hôte	`vagrant reload`, et désactivation de obs pendant le développement
PORGO	Après une bascule réussie, le service restait en 503 : `/sante` dépassait le délai d'une seconde de la sonde, car libpq essaie d'abord bdd1, éteinte	Délai de lecture porté à 3 secondes (section 3.1)
PORGO	Rallumée après une bascule, bdd1 redémarrait en primaire, et rejouer l'installation de bdd2 effaçait la base promue	Le script `reintegrer_bdd.sh` compare les lignes de temps de PostgreSQL et reconvertit l'ancien primaire en réplique (section 5.3)
PORGO	Lors de la validation finale, `vagrant up` échouait sur obs : le dépôt `apt.grafana.com` était injoignable depuis notre réseau	Grafana 13.2.4 est installé depuis le paquet de `dl.grafana.com`, vérifié par son empreinte et gardé en cache dans `.cache/`
KOBENDE	Son Mac M2 (processeur ARM) ne peut pas faire tourner la box	`app.py` a été testé sans base sur son ordinateur, puis dans les machines virtuelles sur le poste de PORGO
THIOMBIANO	Grafana 13 mettait 10 minutes à démarrer sur 768 Mo, car il chargeait une douzaine de plugins dans des processus séparés	Désactivation des plugins inutiles : démarrage en 1 min 08 s (section 3.5)
THIOMBIANO	Grafana affichait comme hors service les machines détruites volontairement lors du retour à deux serveurs	États lus en instantané et frise d'historique des pannes
THIOMBIANO	Pendant V3, 41 % puis 53 % des réponses étaient des 503 sous 50 clients simultanés, et le débit utile baissait avec quatre serveurs	La cause est bdd1, saturée par une connexion authentifiée à chaque requête. Le résultat est expliqué en section 6.3 et PgBouncer est proposé en section 7